Lab 7: Fuzzing for Input Validation Bugs

Name	Aryan Musale
SRN	PES1UG24CS085
Repo	L6_L7_27 — Traffic Fine Calculator (`traffic_fine.py`)
Tools	Python 3.9.6, pytest 8.4.2, Hypothesis 6.141.1

In my repo the application script is `traffic_fine.py` and the test file is `test_traffic_fine.py` (these correspond to `processor.py` / `test_processor.py` in the handout).

Step 1: Installing dependencies

`pip install hypothesis` and `pip install pytest` were installed in the project virtual environment (versions above).

Step 2: Running the buggy application

With the example inputs the program works, but as soon as the ticket ID is missing a hyphen it crashes with an unhandled `ValueError`.

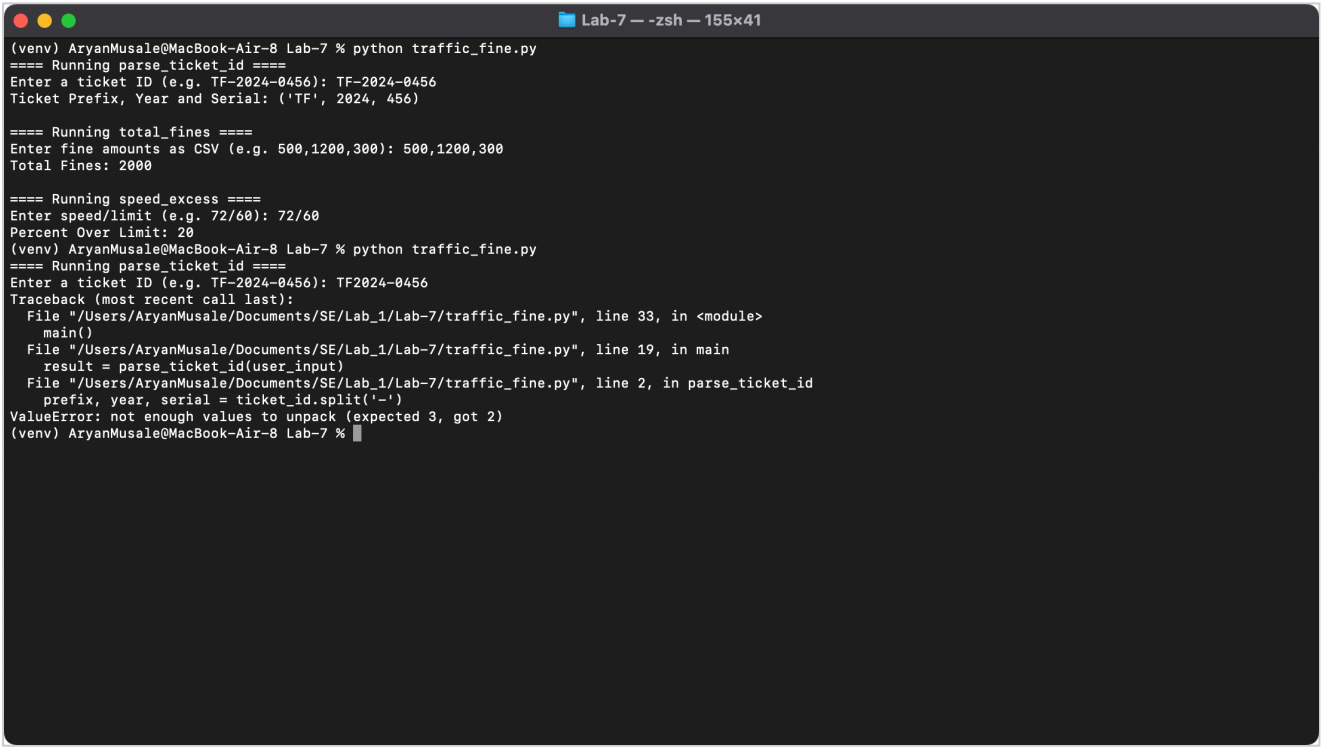


Figure 1 – python traffic_fine.py (original code): valid run, then a crash on input TF2024-0456

Step 3: Hypothesis tests

For each of the three functions I wrote three property-based tests (9 in total):

- **no_crash** – fed `st.text() | st.none()`; the function must never raise.
- **near-valid** – inputs that look like real tickets / CSV lists / speed readings but include blanks, zeros, negatives and extra separators (`st.from_regex`, `st.lists`).
- **valid** – correctly formed inputs; the result must equal the expected value (e.g. total = min(sum, 10000), percent over limit is never negative).

Full code is in the appendix and in `test_traffic_fine.py`.

Step 4: Running the tests on the buggy code

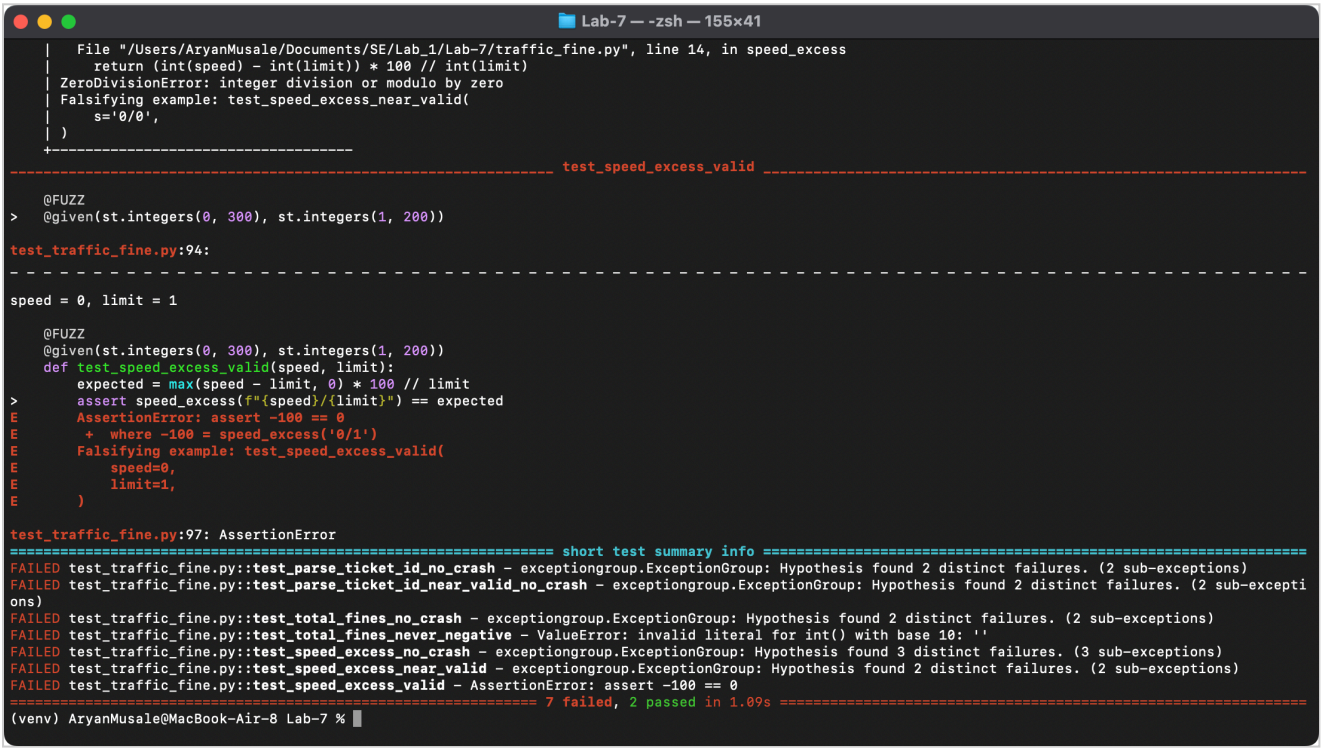


Figure 2 – pytest test_traffic_fine.py (original code): 7 failed, 2 passed, with Hypothesis falsifying examples

Bugs found from the Hypothesis falsifying examples (and the same inputs re-checked by hand):

Function	Falsifying input	Result before fix	Fix
`parse_ticket_id`	`None`	AttributeError: 'NoneType' has no attribute 'split'	Return None for non-string input
`parse_ticket_id`	`'' / 'TF2024–0456' / '---'`	ValueError: wrong number of values to unpack / invalid int literal	Validate full format with an ASCII regex before unpacking
`total_fines`	`None`	AttributeError on .split	Return None for non-string input
`total_fines`	`'' / '500,,300'`	ValueError: invalid literal for int() with base 10: ''	Every comma-separated part must be 1-9 ASCII digits

`total_fines`	`'-5000'`	Negative total accepted silently	Negative amounts rejected (regex allows digits only)
`speed_excess`	`None / '' / '/'`	AttributeError / ValueError	Require exactly two numeric parts
`speed_excess`	`'0/0', '72/0'`	ZeroDivisionError	Return None when limit is 0
`speed_excess`	`'0/1'`	Returns -100 (negative 'percent over limit')	Clamp at 0 when driver is not over the limit

Step 5: Fixed application

Each function now checks its input type, validates the format with an ASCII-only regular expression (so Unicode digits such as '²' are rejected), and returns `None` for bad input instead of raising. Numbers are limited to 9 digits so huge inputs cannot slow the program down. `main()` prints a friendly message when a function returns `None`.

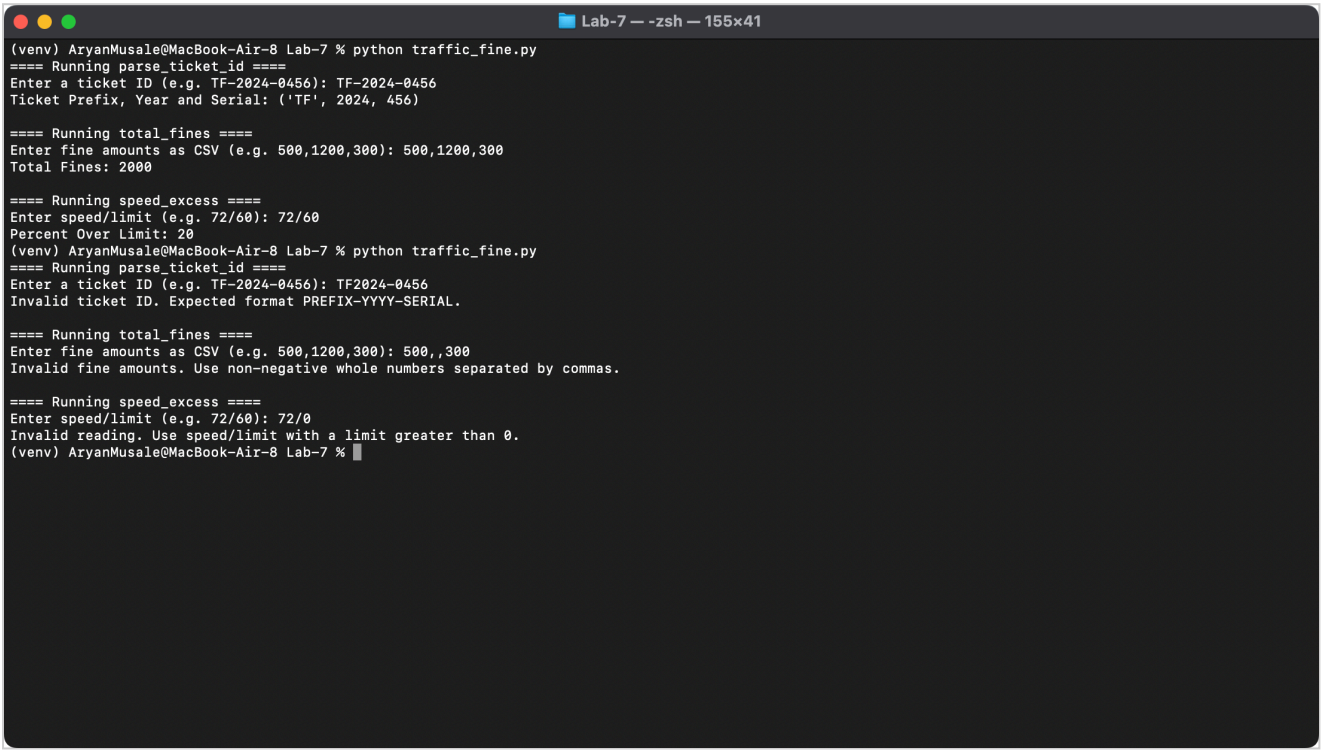


Figure 3 – python traffic_fine.py (fixed code): valid inputs still work; invalid inputs give error messages instead of crashing

Step 6: Re-running the tests on the fixed code

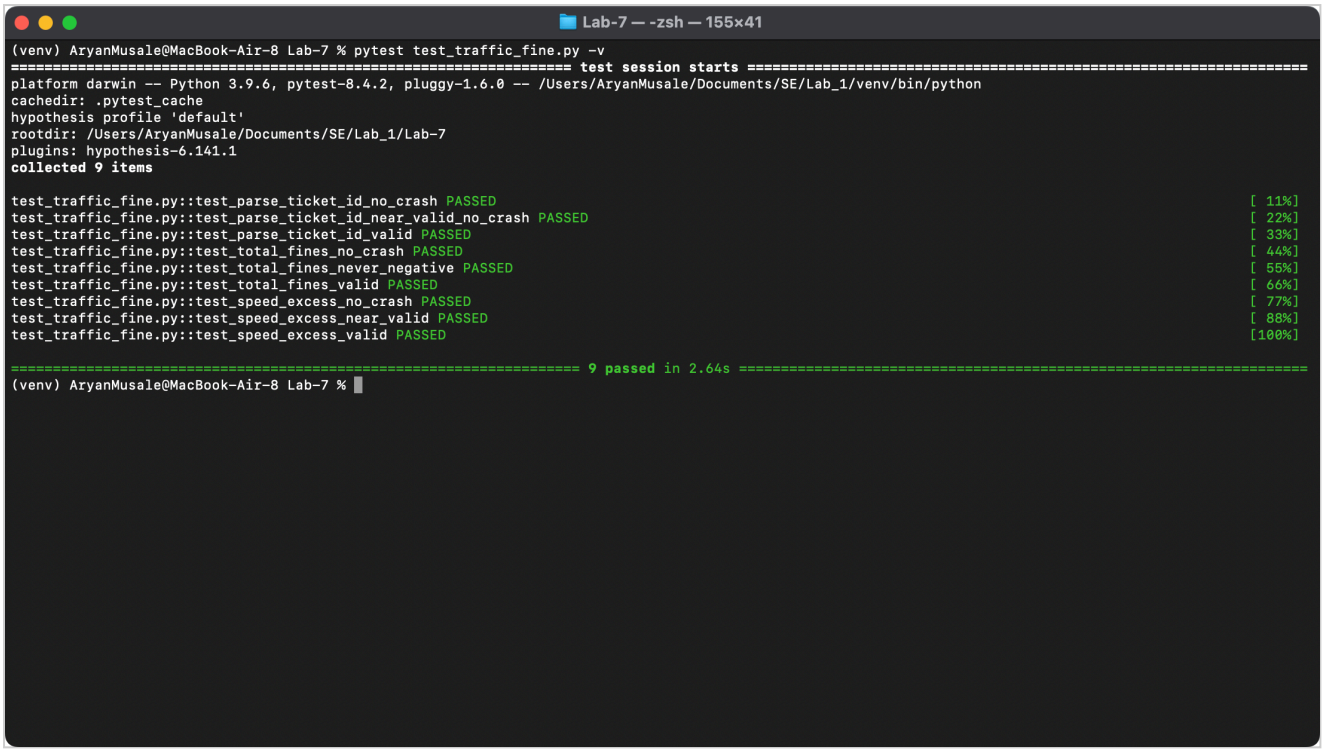


Figure 4 – pytest test_traffic_fine.py -v (fixed code): all 9 tests pass

Appendix A: Fixed traffic_fine.py

```
"""Traffic fine helpers - hardened after fuzz testing (Lab 7).

Every function now validates its input and returns None for anything
malformed instead of raising an exception.
"""
import re

MAX_TOTAL = 10000
MAX_DIGITS = 9  # keeps int() conversions small and fast

# ASCII-only patterns: str.isdigit() / \d would also accept things like '²'
# or Arabic-Indic digits, which int() then rejects or silently converts.
_TICKET_RE = re.compile(r"([A-Za-z]{1,10})-([0-9]{4})-([0-9]{1,%d})" % MAX_DIGITS)
_AMOUNT_RE = re.compile(r"[0-9]{1,%d}" % MAX_DIGITS)


def parse_ticket_id(ticket_id):
    """'TF-2024-0456' -> ('TF', 2024, 456); None if the ID is malformed."""
    if not isinstance(ticket_id, str):
        return None
    match = _TICKET_RE.fullmatch(ticket_id.strip())
    if match is None:
        return None
    prefix, year, serial = match.groups()
    return prefix.upper(), int(year), int(serial)


def total_fines(amounts_csv):
    """'500,1200,300' -> 2000, capped at MAX_TOTAL; None if any amount is bad."""
    if not isinstance(amounts_csv, str):
        return None
    parts = [part.strip() for part in amounts_csv.split(',')]
    if not all(_AMOUNT_RE.fullmatch(part) for part in parts):
        return None  # empty, negative, non-numeric or absurdly long amount
    return min(sum(int(part) for part in parts), MAX_TOTAL)


def speed_excess(reading):
    """'72/60' -> 20 (percent over the limit); 0 if not speeding.

    None if the reading is malformed or the limit is zero.
    """
    if not isinstance(reading, str):
        return None
    parts = [part.strip() for part in reading.split('/')]
    if len(parts) != 2 or not all(_AMOUNT_RE.fullmatch(part) for part in parts):
        return None
    speed, limit = int(parts[0]), int(parts[1])
    if limit == 0:
        return None
    return max(speed - limit, 0) * 100 // limit


def main():
    print("==== Running parse_ticket_id ====")
    user_input = input("Enter a ticket ID (e.g. TF-2024-0456): ")
    result = parse_ticket_id(user_input)
    if result is None:
        print("Invalid ticket ID. Expected format PREFIX-YYYY-SERIAL.")
    else:
        print("Ticket Prefix, Year and Serial:", result)
```

```
    print("\n==== Running total_fines ====")
    user_input = input("Enter fine amounts as CSV (e.g. 500,1200,300): ")
    result = total_fines(user_input)
    if result is None:
        print("Invalid fine amounts. Use non-negative whole numbers separated by commas.")
    else:
        print("Total Fines:", result)

    print("\n==== Running speed_excess ====")
    user_input = input("Enter speed/limit (e.g. 72/60): ")
    result = speed_excess(user_input)
    if result is None:
        print("Invalid reading. Use speed/limit with a limit greater than 0.")
    else:
        print("Percent Over Limit:", result)


if __name__ == "__main__":
    main()
```

Appendix B: test_traffic_fine.py

```
"""Lab 7 - Fuzz testing traffic_fine.py with Hypothesis.

Aryan Musale (PES1UG24CS085)

Each function gets:
  * a "no crash" fuzz test fed with arbitrary text / None, and
  * property tests fed with well-formed or almost-well-formed inputs,
    checking the result is actually correct.
"""
from hypothesis import given, settings, strategies as st

from traffic_fine import parse_ticket_id, total_fines, speed_excess

FUZZ = settings(max_examples=500, deadline=None)

# Inputs that look like the real thing but hit the edge cases
# (zeros, negatives, blanks, extra separators, unicode digits, spaces).
ticket_like = st.from_regex(r"\A[A-Za-z0-9 ]{0,3}-[0-9-]{0,5}-[0-9 ]{0,4}\Z")
csv_like = st.lists(st.integers(-20000, 20000).map(str) | st.just(""),
                    min_size=1, max_size=6).map(",".join)
reading_like = st.from_regex(r"\A-?[0-9]{1,3}/-?[0-9]{1,3}\Z")


# ---------------------------------------------------------------- parse_ticket_id
@FUZZ
@given(st.text() | st.none())
def test_parse_ticket_id_no_crash(s):
    try:
        parse_ticket_id(s)
    except Exception as e:  # pylint: disable=broad-except
        assert False, f"parse_ticket_id crashed with: {e!r}"


@FUZZ
@given(ticket_like)
def test_parse_ticket_id_near_valid_no_crash(s):
    try:
        parse_ticket_id(s)
    except Exception as e:  # pylint: disable=broad-except
        assert False, f"parse_ticket_id crashed with: {e!r}"


@FUZZ
@given(st.from_regex(r"\A[A-Za-z]{1,4}\Z"),
       st.integers(1900, 2100),
       st.integers(0, 99999))
def test_parse_ticket_id_valid(prefix, year, serial):
    ticket = f"{prefix}-{year}-{serial:04d}"
    assert parse_ticket_id(ticket) == (prefix.upper(), year, serial)


# ---------------------------------------------------------------- total_fines
@FUZZ
@given(st.text() | st.none())
def test_total_fines_no_crash(s):
    try:
        total_fines(s)
    except Exception as e:  # pylint: disable=broad-except
        assert False, f"total_fines crashed with: {e!r}"
```

```
@FUZZ
@given(csv_like)
def test_total_fines_never_negative(s):
    result = total_fines(s)
    assert result is None or 0 <= result <= 10000


@FUZZ
@given(st.lists(st.integers(0, 5000), min_size=1, max_size=10))
def test_total_fines_valid(amounts):
    csv = ",".join(str(a) for a in amounts)
    assert total_fines(csv) == min(sum(amounts), 10000)


# ---------------------------------------------------------------- speed_excess
@FUZZ
@given(st.text() | st.none())
def test_speed_excess_no_crash(s):
    try:
        speed_excess(s)
    except Exception as e:  # pylint: disable=broad-except
        assert False, f"speed_excess crashed with: {e!r}"


@FUZZ
@given(reading_like)
def test_speed_excess_near_valid(s):
    result = speed_excess(s)
    assert result is None or result >= 0


@FUZZ
@given(st.integers(0, 300), st.integers(1, 200))
def test_speed_excess_valid(speed, limit):
    expected = max(speed - limit, 0) * 100 // limit
    assert speed_excess(f"{speed}/{limit}") == expected
```